ITAS191 — Lab 4

Design fundamentals and UI patterns

> **Time:** one class, Tuesday 9:00–2:00 · **Marks:** 20

> **Today your site gets a visual system** — type, colour and space chosen on purpose — and a 3D scene you direct. **Assignment 1 is issued today** (the brief is in your bundle), and everything you build in this lab carries straight into it.

Work in `~/itas191/week04/`. Copy last week's `layout.html` and `layout.css` in — today you redesign them — and download the week04 bundle from the Portal for `scene-prompt.md` and the Assignment 1 brief.

How the day runs

Time		
9:00–9:40	**Intro A**	Type, colour, space — a vocabulary for looking good
9:40–10:30	**Session A — Parts 1–2**	~50 min hands-on
10:30–11:00	Break	
11:00–11:40	**Intro B**	Hierarchy, states, motion — and a scene you direct
11:40–12:30	**Session B — Parts 3–6**	Instructor breaks for lunch around noon
12:30–2:00	**Finish + demos**	Help, then **show+tell marking** as you finish

You are marked in person. 6 of the 20 marks are for explaining your choices: why that pairing, what your contrast ratio is and how you know, where keyboard focus goes, what you asked the assistant for and what you changed after. Design is decisions — and a decision you cannot explain was not yours.

Part 1 — A type pairing · 20 min

Go to https://fonts.google.com — no account needed. Choose **two** faces, one job each:

- a **display** face for headings — it carries the personality;
- a **body** face for paragraphs — it carries the reading.

Pair faces that **contrast in kind** — a sans with a serif, a characterful display with a quiet body — never two that are nearly the same. Select both, and Google gives you a `<link>` to paste into your `<head>`, **above** your own stylesheet:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Bricolage+Grotesque:wght@700&family=Source+Serif+4:wght@400;700&display=swap"
rel="stylesheet">
```

Then use them, each with a fallback for when the font cannot load:

```
h1, h2, h3 {
    font-family: "Bricolage Grotesque", sans-serif;
}

body {
    font-family: "Source Serif 4", Georgia, serif;
    line-height: 1.6;
}
```

Now a **type scale** — sizes that relate to each other, not three numbers picked separately. Each step roughly one and a half times the last:

```
body { font-size: 1.1rem; }
h2   { font-size: 1.6rem; }
h1   { font-size: 2.6rem; }
```

`rem` is relative to the reader's own browser setting, so someone who needs bigger text gets all of it bigger, in proportion. Finally, keep lines readable — about 60–75 characters — by giving your paragraphs `max-width: 65ch;`.

> **Hand in:** your layout using two Google Fonts and a visible three-step scale.

Part 2 — A palette that passes · 30 min

A palette is not a list of colours you like. It is **four or five colours with jobs**:

Job	Used for
paper	the page background
ink	body text
accent	links, buttons — the one colour that says "click"
muted	captions, borders, quieter text
surface	section and card backgrounds (optional)

Find yours at https://color.adobe.com or https://coolors.co (no account needed to explore either), or pick them from a photograph you love with the DevTools colour picker. Then write the palette **at the top of `layout.css`**, as a comment — it is a working tool, and in Part 4 you will paste it into a prompt:

```
/* Palette                              contrast on paper
   paper    #f2f5f3   background
   ink      #1b2a2f   body text           13.48:1
   accent   #b23a48   links, buttons       5.33:1
   muted    #5f7470   captions, borders    4.53:1
   surface  #ffffff   section backgrounds
*/
```

Contrast is a number, not an opinion. Check every text colour against the background it sits on at https://webaim.org/resources/contrastchecker/ — no account. Body text must be at least **4.5:1**; that is the accessibility standard (WCAG AA), and it is what a reader with low vision, or anyone on a phone in sunlight, needs. Record each ratio in your palette comment. Notice `muted` above: it passes at 4.53 — by a hair. Muted colours are where palettes fail. Check, never guess.

Apply the palette across the whole page: the body's background and text, the header, every link, the section surfaces.

> **Hand in:** the palette comment with jobs and recorded ratios, applied, every text-on-background pair at 4.5:1 or better.

Part 3 — A header and nav that look designed · 35 min

Hierarchy first. Step back from your screen and squint. What do you see first? It should be your site's name, then the navigation, then the content. If something else wins, fix it with **size, weight and space** — not with more colours.

A nav that looks like navigation. Your nav is already a flex container (week 3). Make it a row, give the links room, and style them as controls, not as underlined text:

```
nav {
    display: flex;
    gap: 1.5rem;
}

nav a {
    color: #1b2a2f;
    text-decoration: none;
    padding: 0.3rem 0;
    transition: color 0.2s;
}
```

Two states, both required. A link needs to answer "can I click this?" for a mouse and for a keyboard:

```
nav a:hover,
nav a:focus {
    color: #b23a48;
    text-decoration: underline;
}
```

`:hover` is the mouse. `:focus` is the keyboard — and some of your visitors never touch a mouse. **Test it:** click in the address bar, then press Tab repeatedly. Every link in your nav must visibly light up as focus reaches it. If your CSS (or an assistant's) contains `outline: none`, that is what is hiding the focus ring — delete it, or replace it with something just as visible.

The `transition` on `nav a` is what makes the colour change glide instead of snap. Motion should be **feedback** — it tells you something happened. Keep it short; 0.2 seconds is plenty. Put it on the link's normal state, not only on `:hover`, so it glides both ways.

Finally: name what you built. Is your nav a nav bar? Is your header a hero? Write the pattern names as a comment in your CSS.

> **Hand in:** a redesigned header and nav, with hover and visible keyboard focus, and a transition.

Part 4 — A 3D scene you direct · 40 min

Last week you placed a cube someone else wrote. This week the scene is yours: you **write the brief**, an assistant writes the code, and you **art-direct** the result until it belongs to your site.

1. **Fill in the brief.** Open `scene-prompt.md` from the bundle. It is a prompt with blanks: your palette's hex values, the size of the frame, what the scene shows, and how it should move. The other rules in it — one version of Three.js, from one place, drawing into one `<div>` — are fixed. They are what make the answer paste cleanly into your page, and they stop the assistant pulling in scripts from anywhere else.
2. **Send it.** Give the whole filled-in prompt to Claude, or whichever assistant you use. It answers with three pieces: a CSS rule for `#scene`, the `<div id="scene">`, and the `<script>` blocks.
3. **Place it** — the same three-piece paste as week 3's cube, into a column of your layout. **One import map per page:** if week 3's cube is still in this file, take it out first.
4. **Art-direct it.** The first answer will look like everyone else's: a spinning shape, default colours, a void behind it. Make at least **two** changes so the scene belongs to your site — a shape that means something to your content, motion that suits your mood, a better camera angle, your palette in every surface and light. Ask the assistant for each change, or find the commented line and change it yourself.
5. **Keep the record.** In `prompts.md`, write down every prompt you sent, in order, and one line under each saying what changed. That file is your AI disclosure for this part.

What you are and are not answerable for. You are this scene's **art director**, not its programmer — JavaScript is week 8. At the demo you will be asked what you asked for, what you changed, and **where in the code your colours live**. You will not be asked what line 30 does.

A word about what you just did. Your page now runs code you did not write. That is normal — every library works that way — and it is exactly why the prompt pins one version from one source. Read the comments before you paste.

> **Hand in:** the scene placed in your layout, in your palette, with two art-directed changes; and `prompts.md`.

Part 5 — Name the pattern · 15 min

Open three real sites you actually use. In each, find one UI pattern from this morning — nav bar, hamburger menu, breadcrumbs, card, hero, footer — and in `patterns.md` write three lines for each:

- the site, and the pattern;

- what it is doing for the visitor;
- **does it earn its place**, or is it there because every site has one?

This is the vocabulary for the critique ritual, which starts today: one site, on the projector, every week — what works, what does not, one change.

> **Hand in:** `patterns.md` with three entries.

Part 6 — Validate, then start Portfolio v1 · 30 min

Both validators, both files — HTML at https://validator.w3.org, CSS at https://jigsaw.w3.org/css-validator/. Zero errors. The scene's script blocks validate as written; if the HTML fails only there, the paste went wrong.

Then open the **Assignment 1 brief** in your bundle and start it while you can still ask questions in the room. Make `~/itas191/a1/`, and carry your fonts and palette across — Portfolio v1 is built on the visual system you made today.

> **Hand in:** both files validating clean.

Marking — 20 marks

	Component	Marks
C1	Type — two Google Fonts paired, a visible three-step scale	2
C2	Palette — jobs documented, ratios recorded, every text pair at 4.5:1 or better	3
C3	Header and nav — clear hierarchy, hover and visible focus, a transition	3
C4	The 3D scene — inside the fixed rules, in your palette, two art-directed changes, `prompts.md`	3
C5	Three patterns named and judged	1
C6	Both validators clean	2
DU	**Demonstrates understanding** — marked in person	**6**
		20

`DU` comes from your demo. Expect: "why these two faces?", "what is your body text's contrast ratio — show me how you know", "Tab to your nav: where is focus?", "what did you ask for, and what did you change after?", "find the line that sets this colour."

Submitting

Demo first, then upload your `week04` folder to the Portal: `layout.html`, `layout.css`, `prompts.md` and `patterns.md`.

On using AI for this lab

Permitted and encouraged, as since week 3 — **disclose it, own it.** One line at the bottom of each file says what you asked for; for the scene, `prompts.md` is the disclosure.

Owning it has two meanings this week. For your **type and palette**, an assistant may suggest — but the ratios are checked by you, and at the demo you say why those colours. For the **scene**, owning it means owning the direction: the brief, the changes, and knowing where the colours live in the code.

If something goes wrong

What you see	What it means	What to do
Your Google Font does not appear	The font `<link>` is missing, below your own stylesheet, or the name in CSS does not match exactly	Put the font link first. Copy the family name from Google exactly, in quotes if it has a space.
The font works here but not on another machine	The font loads over the internet; offline, the fallback takes over	That is what the fallback is for. Make sure you have one.
The contrast checker says fail	Text and background are too close in lightness	Darken the text or lighten the background. Change lightness, not hue. Muted colours fail first.
No focus ring when you press Tab	Something set `outline: none` — often an assistant, or a copied reset	Delete it, or replace it with a visible style of your own.
The transition only works one way	It is on `:hover` alone	Move `transition` to the link's normal state.
The scene is a blank space	The import map is missing, after the module script, or doubled — **one import map per page**	F12 → **Console**. It names the line.
The scene is the wrong size	The `#scene` rule's width and height do not fit your column	Change the two numbers in that rule.
The assistant used a different Three.js	It ignored rule 1 of the prompt	Paste the import map back and say: use exactly this, nothing else.

Same instrument as every week: DevTools shows you what the browser actually did, and the Console says why a script did not run.